



Lista de Exercícios
Funções Hash: Teoria, SHA-2 e SHA-3
(Com Respostas)

Prof. Iaçanã Ianiski Weber

Exercícios adaptados de Paar, Pelzl e Güneysu, *Understanding Cryptography*, 2ª ed., Springer, 2024 (Capítulo 11).

Observação: esta versão contém os enunciados e uma proposta de solução. Em itens discursivos, respostas equivalentes podem ser igualmente corretas.

Exercício 1

Sistemas de autenticação costumam armazenar $h(\text{senha})$ em vez da senha: o valor digitado é processado por h e comparado com o hash guardado. Suponha que um atacante obteve o arquivo com todos os hashes de senha. (Livro, Problema 11.5)

- (a) Para cada cenário abaixo, discuta se o ataque permite recuperar senhas para se passar por usuários legítimos e com quais consequências: **Ataque A:** quebrar a propriedade de mão única (*one-wayness*) de h ; **Ataque B:** encontrar segundas pré-imagens de h ; **Ataque C:** encontrar colisões de h .
- (b) Por que essa técnica costuma ser estendida com um *salt* (valor aleatório anexado à senha antes do hash)? Os ataques do item (a) são afetados pelo *salt*?
- (c) Uma função hash com saída de **80 bits** é suficiente para essa aplicação? Justifique com base nas propriedades que ela realmente exige.

(a) Ataque A — é o ataque devastador: invertendo h , o atacante recupera diretamente a senha correspondente a cada hash da lista e consegue se passar por *qualquer* usuário do sistema.

Ataque B — encontrar uma segunda pré-imagem significa: *dado um par conhecido* $(x_1, h(x_1))$, achar $x_2 \neq x_1$ com $h(x_2) = h(x_1)$. O atacante, porém, só possui os hashes — ele não conhece nenhuma senha x_1 dos outros usuários. O ataque só é útil se ele já tiver uma senha válida que produza o hash-alvo (e, nesse caso, ele já teria o acesso). Logo, não permite recuperar senhas a partir da lista.

Ataque C — numa colisão, o atacante escolhe livremente *ambas* as mensagens $x_1 \neq x_2$ com $h(x_1) = h(x_2)$. Nada garante que algum desses valores coincida com um hash já armazenado na lista; portanto, colisões não ajudam a personificar usuários existentes.

(b) Sem *salt*, dois usuários com a mesma senha têm o mesmo hash: o atacante detecta senhas repetidas e, pior, pode usar tabelas pré-computadas (ex.: *rainbow tables*) válidas para todos os usuários de todos os sistemas. Com um *salt* aleatório e individual, $h(\text{senha} \parallel \text{salt})$ quebra essa relação determinística e obriga o atacante a atacar cada conta separadamente. Os três ataques genéricos do item (a), entretanto, *não são afetados* em sua essência: o *salt* é armazenado em claro junto ao hash, então ele não aumenta a resistência a pré-imagem de uma conta específica — apenas impede ataques em lote/pré-computados.

(c) Sim. A aplicação exige apenas **mão única** (e resistência a segunda pré-imagem); resistência a colisões *não* é necessária, pois colisões não comprometem senhas armazenadas (item a). Como o ataque do aniversário não se aplica, a segurança efetiva é de 2^{80} passos (busca de pré-imagem),

e não 2^{40} . *Nota:* 2^{80} é considerado o limiar mínimo hoje; em sistemas novos recomenda-se saída maior (e funções lentas dedicadas a senhas, como bcrypt/Argon2, cf. Seção 11.6 do livro).

Exercício 2

Considere três funções hash com saídas de 64, 128 e 160 bits. O número esperado de entradas aleatórias que precisam ser processadas para que ocorra uma colisão com probabilidade λ é aproximadamente

$$t \approx 2^{(n+1)/2} \sqrt{\ln\left(\frac{1}{1-\lambda}\right)},$$

onde n é o comprimento da saída em bits. (Livro, Problema 11.3)

- (a) Para cada um dos três comprimentos, após quantas entradas aleatórias há probabilidade $\lambda = 0,5$ de colisão?
- (b) E para $\lambda = 0,1$?

Para $\lambda = 0,5$: $\sqrt{\ln 2} \approx 0,8326$; para $\lambda = 0,1$: $\sqrt{\ln(1/0,9)} \approx 0,3246$. Substituindo na equação (11.1) do livro:

n	$\lambda = 0,5$	$\lambda = 0,1$
64	$t \approx 2^{32,2} \approx 5,1 \cdot 10^9$	$t \approx 2^{30,9} \approx 2,0 \cdot 10^9$
128	$t \approx 2^{64,2} \approx 2,2 \cdot 10^{19}$	$t \approx 2^{62,9} \approx 8,5 \cdot 10^{18}$
160	$t \approx 2^{80,2} \approx 1,4 \cdot 10^{24}$	$t \approx 2^{78,9} \approx 5,5 \cdot 10^{23}$

Observe que reduzir a probabilidade desejada de 50% para 10% diminui t em menos de 2 bits de expoente: a complexidade do ataque do aniversário é essencialmente $t \approx 2^{n/2}$, pouco sensível a λ .

Nota para o professor: o manual oficial de soluções (2ª ed.) lista valores menores por um fator $\sqrt{2}$ (ex.: $3,6 \cdot 10^9$ para $n = 64$, $\lambda = 0,5$), pois omite o “+1” do expoente da própria equação (11.1) do livro. Os valores acima seguem a equação como enunciada (e como consta no exercício).

Exercício 3

Vamos computar à mão a primeira rodada ($j = 0$) da função de compressão do SHA-256. Ignore os valores iniciais padronizados e assumamos todos os registradores zerados: $A_0 = \dots = H_0 = 00000000_{\text{hex}}$. Lembre que $T_1 = H + \Sigma_1(E) + Ch(E, F, G) + K_j + W_j$, $T_2 = \Sigma_0(A) + Maj(A, B, C)$, e que $K_0 = 428A2F98_{\text{hex}}$. (Livro, Problema 11.9)

- (a) Calcule os registradores $A_1, \dots, H_1$ para o bloco $x = \{0 \dots 00\}$ (todas as palavras W_j nulas).
- (b) Repita para $x = \{0 \dots 01\}$, isto é, $W_0 = 00000001_{\text{hex}}$ e as demais palavras nulas.
- (c) O que (a) e (b) sugerem sobre o papel das constantes K_j quando a entrada é toda nula?

Com todos os registradores zerados, todas as funções internas se anulam: $\Sigma_1(0) = 0$ (rotações de zero), $Ch(0, 0, 0) = (0 \wedge 0) \oplus (\neg 0 \wedge 0) = 0$, $\Sigma_0(0) = 0$ e $Maj(0, 0, 0) = 0$. Logo:

$$T_1 = H_0 + \Sigma_1(E_0) + Ch(E_0, F_0, G_0) + K_0 + W_0 = K_0 + W_0, \quad T_2 = \Sigma_0(A_0) + Maj(A_0, B_0, C_0) = 0.$$

A atualização dos registradores é $A_1 = T_1 + T_2$, $E_1 = D_0 + T_1$ e os demais apenas deslocam: $B_1 = A_0$, $C_1 = B_0$, $D_1 = C_0$, $F_1 = E_0$, $G_1 = F_0$, $H_1 = G_0$.

(a) Com $W_0 = 00000000$: $T_1 = K_0 = 428A2F98$, então

$$A_1 = E_1 = 428A2F98_{\text{hex}}, \quad B_1 = C_1 = D_1 = F_1 = G_1 = H_1 = 00000000_{\text{hex}}.$$

(b) Com $W_0 = 00000001$: $T_1 = K_0 + W_0 = 428A2F99$, então

$$A_1 = E_1 = 428A2F99_{\text{hex}}, \quad B_1 = C_1 = D_1 = F_1 = G_1 = H_1 = 00000000_{\text{hex}}.$$

(c) Sem as constantes K_j , uma entrada toda nula com estado todo nulo produziria $T_1 = T_2 = 0$ e o estado permaneceria nulo *em todas as rodadas* — a função de compressão teria um “ponto fixo” trivial e degenerado. As constantes de rodada quebram essa simetria e garantem que mesmo entradas degeneradas (todas nulas, todas em 1, padrões repetitivos) sejam misturadas; é o mesmo papel que as constantes $RC[i]$ da etapa ι desempenham no Keccak- f .

Exercício 4

Use uma ferramenta de linha de comando para calcular o hash SHA-512 das mensagens $m_1 = \text{Have you eaten, my child?}$ e $m_2 = \text{Have you eaten my child?}$. Calcule o hash de cada mensagem *sem* o caractere de nova linha ao final; no Linux, por exemplo: `printf 'Have you eaten, my child?' | sha512sum` (Livro, Problema 11.10)

- Apresente os quatro primeiros bytes de cada hash em notação hexadecimal.
- As mensagens m_1 e m_2 são quase idênticas (diferem em uma vírgula). O que você observa nos hashes? Que nome damos a esse comportamento e por que ele é desejável?

(a) Sem o caractere de nova linha ao final:

$$\text{SHA-512}(m_1) = 81\ 9F\ 6E\ 39\dots \quad \text{SHA-512}(m_2) = E7\ 82\ FE\ 26\dots$$

Nota para o professor: se o aluno usar `echo` sem `-n`, um `\n` é anexado à mensagem e os hashes mudam completamente — vale aceitar a resposta se o procedimento estiver coerente, mas é uma boa oportunidade para discutir que o hash é sobre *bytes exatos*, não sobre o “texto”.

(b) Os dois hashes são completamente diferentes, sem qualquer semelhança visível, embora as mensagens difiram em um único caractere. Esse comportamento é o **efeito avalanche** (difusão): a alteração de 1 bit da entrada inverte, em média, metade dos bits da saída. Ele é desejável porque o hash não deve vaziar informação sobre a entrada — em particular, mensagens parecidas não podem produzir hashes parecidos, senão um atacante poderia inferir conteúdo ou procurar colisões por aproximação.

Exercício 5

O Keccak- f é uma **função de permutação**: cada um dos 2^b valores possíveis do estado é mapeado, de forma bijetiva (um-para-um), para exatamente um valor de saída. Isso é diferente de uma **permutação de bits** (como as permutações P e IP do DES), que apenas troca as *posições* dos bits. Para enxergar a diferença, considere um exemplo em miniatura com $d = 2$ bits de entrada e saída. (Livro, Problema 11.13)

- (a) Quantas permutações de bits existem com 2 bits de entrada e saída? Desenhe um diagrama para cada uma.
- (b) Quantos valores distintos de entrada (e de saída) existem? E quantas funções de permutação (mapeamentos bijetivos) distintas existem entre entrada e saída? Justifique com um argumento de contagem.
- (c) Dentre as funções de permutação do item (b), quais correspondem às permutações de bits do item (a)? Mostre as duas tabelas-verdade.
- (d) Generalize: para d bits, quantas permutações de bits e quantas funções de permutação existem? O que esses dois números sugerem no caso do estado do Keccak- f ($b = 1600$)?

(a) Existem apenas $2! = 2$ permutações de bits: a *identidade* (bit 1 $\rightarrow$ bit 1, bit 2 $\rightarrow$ bit 2) e o *cruzamento* (bit 1 $\rightarrow$ bit 2, bit 2 $\rightarrow$ bit 1):



(b) Há $2^2 = 4$ valores de entrada e 4 de saída: $\{00, 01, 10, 11\}$. Contagem das bijeções: a primeira entrada pode ser mapeada para 4 saídas; a segunda, para 3 restantes; a terceira, para 2; a última fica determinada. Total: $4 \cdot 3 \cdot 2 \cdot 1 = 4! = \mathbf{24}$ funções de permutação.

(c) Apenas 2 das 24 bijeções são realizáveis trocando posições de bits:

identidade		cruzamento	
entrada	saída	entrada	saída
00	00	00	00
01	01	01	10
10	10	10	01
11	11	11	11

Note que toda permutação de bits fixa 00 e 11 — por isso a maioria das bijeções *não* pode ser obtida apenas reposicionando bits.

(d) Para d bits: $d!$ permutações de bits contra $(2^d)!$ funções de permutação. Para o Keccak- f ($b = 1600$): $1600!$ contra $(2^{1600})!$ — o segundo número é inimaginavelmente maior. Permutações de bits formam uma fração ínfima e estruturalmente pobre (são lineares e não criam *confusão*); já o Keccak- f é projetado para se comportar como uma permutação “genérica” desse espaço gigantesco, o que fundamenta a segurança da construção esponja.

Exercício 6

Suponha que o SHA-3 é usado em uma aplicação com saída de 256 bits (SHA3-256, taxa $r = 1088$ bits) e que medimos uma vazão de **120 MByte/s** com nossa biblioteca criptográfica. Por razões de segurança, decidimos trocar para o SHA3-384, cuja taxa é $r = 832$ bits. (Livro, Problema 11.11)

- (a) Explique por que a vazão muda, considerando como a construção esponja absorve a mensagem em blocos de r bits.
- (b) Calcule a vazão aproximada da nova configuração, assumindo que o custo de cada invocação de Keccak- f é o mesmo nos dois casos.

(a) Na fase de absorção, cada invocação de Keccak- f processa exatamente r bits da mensagem (o estado é sempre de $b = 1600$ bits; o que muda é a parcela r disponível para a mensagem, pois $c = b - r$ cresce com o nível de segurança). Como o custo de Keccak- f independe de r , absorver a mesma mensagem com r menor exige *mais* invocações — daí a queda de vazão.

(b) A vazão é proporcional a r :

$$120 \text{ MByte/s} \times \frac{832}{1088} \approx 91,8 \text{ MByte/s} \approx 92 \text{ MByte/s}.$$

Nota para o professor: o manual oficial de soluções responde 97 MByte/s usando $r = 1344$ e $r = 1088$ — mas essas são as linhas erradas da Tabela 11.3 do livro (1344 é a taxa do SHAKE128; a do SHA3-256 é 1088 e a do SHA3-384 é 832). O cálculo acima segue a tabela do livro (e o padrão FIPS 202).

Exercício 7

Queremos calcular o hash da mensagem curta de dois bytes 0xCCCC usando o SHA-3 como substituto do SHA-2 com 256 bits de saída (SHA3-256, $r = 1088$). Lembre que o SHA-3 anexa o sufixo 01 à mensagem e, em seguida, aplica o *multi-rate padding* 10^*1 até completar um múltiplo de r bits. (Livro, Problema 11.12)

- (a) Qual é a mensagem após sufixo e padding? Apresente a resposta em notação binária, indicando claramente cada campo (mensagem, sufixo, padding) e a quantidade de zeros.
- (b) Quantos blocos de r bits serão absorvidos pela esponja? Quantas invocações de Keccak- f ocorrem na fase de absorção?

(a) A mensagem tem 16 bits: 0xCCCC = 1100 1100 1100 1100. Após anexar o sufixo 01, temos 18 bits. O multi-rate padding 10^j1 deve completar 1088 bits, logo ocupa $1088 - 18 = 1070$ bits: um 1, depois $j = 1068$ zeros, depois o 1 final:

$$\underbrace{1100110011001100}_{\text{mensagem (16 bits)}} \parallel \underbrace{01}_{\text{sufixo}} \parallel \underbrace{1 \underbrace{0 \dots 0}_{1068 \text{ zeros}} 1}_{\text{padding (1070 bits)}} \Rightarrow 16 + 2 + 1070 = 1088 \text{ bits}.$$

(b) O resultado ocupa exatamente **um bloco** de $r = 1088$ bits; portanto, a fase de absorção realiza **uma única invocação** de Keccak- f . (Como a saída desejada, 256 bits, cabe em y_0 ,

também não há invocações extras na fase de espremedura.)

Exercício 8

Considere um estado A do SHA-3 (Keccak- f com $b = 1600$, palavras de $w = 64$ bits) em que todos os bits valem 0, *exceto* os bits cuja coordenada z é igual a zero, isto é, $A[x, y, 0] = 1$ para todos x, y . Lembre que a etapa θ computa

$$A'[x, y, z] = A[x, y, z] \oplus \bigoplus_{y'=0}^4 A[x-1, y', z] \oplus \bigoplus_{y'=0}^4 A[x+1, y', z-1],$$

com índices x módulo 5 e z módulo 64. (Livro, Problema 11.15)

- (a) Quantos bits do estado valem 1? Onde esses bits estão localizados na representação tridimensional do estado (que nome recebe essa “fatia”)?
- (b) Aplique a etapa θ ao estado A e descreva o novo estado A' (analise os bits com $z = 0$, $z = 1$ e $z \geq 2$).

(a) São $5 \times 5 = 25$ bits em 1: todos os bits do plano $z = 0$. Na representação tridimensional do estado, esse plano é chamado de **fatia** (*slice*) — aqui, a primeira fatia.

(b) Analisando $A'[x, y, z] = A[x, y, z] \oplus P(x-1, z) \oplus P(x+1, z-1)$, onde $P(x, z) = \bigoplus_{y'} A[x, y', z]$ é a paridade da coluna (x, z) . No estado dado, toda coluna da fatia $z = 0$ tem cinco 1s, logo $P(x, 0) = 1$ para todo x ; todas as demais fatias têm paridade 0.

• **Bits com $z = 0$:** $A' = \underbrace{1}_{A[x,y,0]} \oplus \underbrace{1}_{P(x-1,0)} \oplus \underbrace{0}_{P(x+1,63)} = 0.$

• **Bits com $z = 1$:** $A' = \underbrace{0}_{A[x,y,1]} \oplus \underbrace{0}_{P(x-1,1)} \oplus \underbrace{1}_{P(x+1,0)} = 1.$

• **Bits com $z \geq 2$:** todos os três termos são 0, logo $A' = 0$.

Resultado: o novo estado A' tem exatamente os 25 bits da fatia $z = 1$ em 1 e todos os demais em 0 — a fatia “acesa” deslocou-se de $z = 0$ para $z = 1$. (O exemplo também ilustra como θ propaga informação entre fatias vizinhas.)